

CS4031 – Compiler Construction

Course Project Proposal – Spring 2026



Theme: Pixel Art Generator • Weightage: 10% • Total Marks: 100

Final Submission Deadline: 10th May 2026

Team Members

Student Name	Student ID
Kinza	23K-0842

1. Language Concept

PixelLang is a retro-inspired mini-language designed to allow users to create pixel-based images and patterns via scripting. Its purpose is to simplify the generation of static and animated pixel art, making it accessible for beginners and enthusiasts of retro graphics.

Users can define pixels, draw shapes, apply transformations, and use loops to create complex patterns efficiently. The language provides a simple syntax that compiles into Python-generated images, producing PNG outputs directly.

2. Key Features

- **Feature 1:** Place individual pixels on a canvas using `PIXEL x y COLOR` statements.
- **Feature 2:** Draw simple geometric shapes (`RECT`, `LINE`, `CIRCLE`) with color support.
- **Feature 3:** Apply transformations such as `TRANSLATE` and `ROTATE` to shapes and loops.
- **Feature 4:** Support loops with `LOOP n { ... }` to repeat patterns or shapes.
- **Feature 5:** Built-in color handling with hexadecimal color codes (`#RRGGBB`) and comment support (`//`).

3. Example Program

```
// Draw a simple red pixel
PIXEL 0 0 #FF0000;

// Draw a green rectangle
RECT 2 2 5 5 #00FF00;

// Loop to draw a blue diagonal pattern
LOOP 3 {
    PIXEL 0 0 #0000FF;
    TRANSLATE 1 1;
}
```

Explanation

- The first statement places a red pixel at the top-left corner (0, 0).
- The `RECT` command draws a green 5×5 shape starting at coordinate (2, 2).
- The `LOOP` block creates a diagonal blue pattern by translating the pixel after each iteration.

4. Target Output

Output Type	Generated image file (PNG) showing the pixel art pattern.
Optional Execution	Live preview of the canvas during compilation.
Direct Results	The compiler interprets the PixelLang script and produces a visual representation of the pixel artwork automatically.

5. Expected Challenges

- **Challenge 1:** Ensuring transformations such as translation and rotation apply correctly across loops and nested statements.
- **Challenge 2:** Validating pixel coordinates, shape dimensions, and color codes to prevent errors or out-of-bounds drawing.
- **Challenge 3:** Designing a parser and AST that can handle nested loops, multiple statements, and different shape commands efficiently.